Lab Assignment No.	07
Title	To apply the artificial immune pattern recognition to perform a task of structure damage Classification.
Roll No.	
Class	BE AI & DS
Date Of Completion	
Subject	Computer Laboratory III[417534]
Assessment Marks	
Assessor's Sign	

Assignment No. 07

Title: Application of Artificial Immune Pattern Recognition for Structural Damage Classification

Problem Statement: To apply artificial immune pattern recognition techniques to perform the task of structural damage classification, utilizing bio-inspired algorithms to detect and classify damage in structural components.

Prerequisites:

- Basic knowledge of Python programming
- Understanding of pattern recognition and classification techniques
- Familiarity with bio-inspired algorithms and artificial immune systems
- Basic knowledge of Jupyter Notebook environment

Software Requirements:

- Jupyter Notebook
- Python 3.x
- Required Python libraries: NumPy, Pandas, Scikit-learn, Matplotlib

Hardware Requirements:

- A computer with minimum 4 GB RAM
- Dual-core processor or higher
- Stable internet connection

Theory: Artificial Immune Systems (AIS) are computational models inspired by the principles and processes of the biological immune system. AIS are particularly effective in pattern recognition tasks, such as structural damage classification, due to their adaptive learning capabilities, robustness, and efficiency in handling complex, dynamic data environments.

The AIS framework operates by simulating the immune system's mechanisms of recognizing, learning, and remembering patterns. In the context of structural damage classification, AIS mimics the immune system’s ability to detect and classify anomalies (damages) based on

extracted structural features. The system utilizes concepts such as antigen-antibody interactions, clonal selection, and affinity maturation to optimize classification performance.

Key Concepts:

1. **Pattern Recognition:** The process of identifying and categorizing patterns within structural health monitoring data to detect potential damage.
2. **Antigen-Antibody Interaction:** Structural features representing potential damage (antigens) are compared with candidate solutions (antibodies) generated by the AIS model.
3. **Clonal Selection Principle:** High-affinity antibodies (accurate classifiers) are selected, cloned, and mutated to improve the system's ability to classify structural conditions.
4. **Affinity Measure:** A quantitative assessment of how well an antibody (classifier) recognizes and classifies a given antigen (damage pattern).
5. **Mutation and Diversity:** Introduces variability within cloned antibodies to explore new solutions and improve generalization capability.
6. **Memory Set:** A collection of the best-performing antibodies that are retained for future classification tasks, enhancing the system’s learning over time.

Mathematical Model:

- **Affinity Calculation:** Measures the similarity between structural features (antigens) and candidate solutions (antibodies) using distance metrics or similarity functions.
- **Classification Rule:** Assigns structural conditions (e.g., damaged or undamaged) based on the highest affinity score.
- **Mutation Operator:** Applies controlled alterations to cloned antibodies to maintain diversity and prevent overfitting.

Applications:

- Structural health monitoring in civil engineering
- Damage detection in bridges, buildings, and aerospace structures
- Mechanical component fault diagnosis
- Monitoring and maintenance planning in critical infrastructure systems

Source Code –

```
# Import Required Libraries

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns from sklearn.model_selection

import train_test_split from sklearn.metrics

import accuracy_score, confusion_matrix, classification_report

#Load and Prepare the Dataset

df = pd.read_csv(r"C:\Users\saira\Downloads\structural_damage_data.csv")

df.head()

df.tail()

df.isnull().sum()

# Feature Selection and Spiltting data

X = df.drop('Damage_Level', axis=1) y = df['Damage_Level']

# Split into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Artificial Immune Pattern Recognition (AIPR) Algorithm

class AIPR:

def __init__(self, n_clones=20, mutation_rate=0.05, generations=100):

 self.n_clones = n_clones

self.mutation_rate = mutation_rate

self.generations = generations

def fit(self, X, y):

self.memory_cells = np.random.rand(self.n_clones, X.shape[1])
```

```
self.labels = np.random.choice(np.unique(y), self.n_clones)
for _ in range(self.generations):
 for i in range(self.n_clones):
clone = self.memory_cells[i] + self.mutation_rate * np.random.randn(X.shape[1])
 clone_fitness = self._fitness(clone, X, y)
if clone_fitness > self._fitness(self.memory_cells[i], X, y):
self.memory_cells[i] = clone
 def predict(self, X):
preds = []
for x in X:
distances = np.linalg.norm(self.memory_cells - x, axis=1)
 pred = self.labels[np.argmin(distances)]
preds.append(pred)
return np.array(preds)
def _fitness(self, antibody, X, y):
distances = np.linalg.norm(self.memory_cells - antibody, axis=1)
closest_label = self.labels[np.argmin(distances)]
return np.mean(closest_label == y)
# Train and Evaluate the Model
# Initialize and train the AIPR model
model = AIPR(n_clones=20, mutation_rate=0.05, generations=100)
model.fit(X_train.values, y_train.values)
# Make predictions
y_pred = model.predict(X_test.values)
# Evaluation
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
# Confusion Matrix
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, cmap='Blues')
 plt.title('Confusion Matrix')
plt.show()
# Visualize Result
# Compare true vs predicted damage levels
plt.figure(figsize=(10, 5))
 plt.plot(y_test.values, label='True')
 plt.plot(y_pred, label='Predicted', alpha=0.7)
plt.legend()
plt.title('True vs Predicted Damage Levels')
plt.show()
```

Conclusion: Through this practical, we have explored the application of artificial immune pattern recognition for structural damage classification. By implementing bio-inspired algorithms in Python, we demonstrated the effectiveness of AIS in identifying and classifying structural damage patterns. This practical highlighted the potential of artificial immune systems in enhancing structural health monitoring and improving the reliability of damage detection processes.